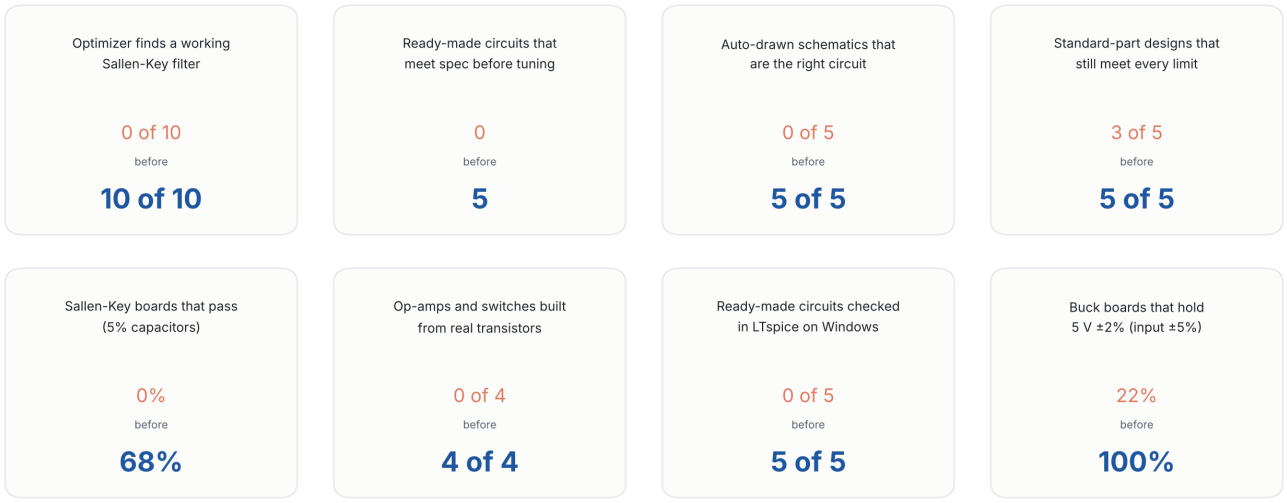
SpiceMCP

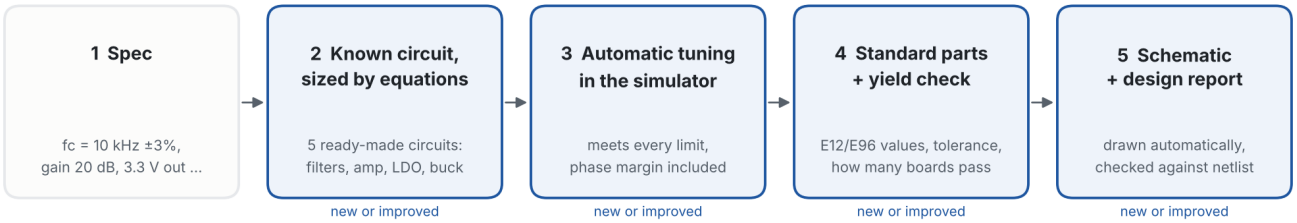
From a circuit-simulation prototype to an engineering design tool

What changed in versions 0.6 to 0.15, shown with simulations, and what it means for someone designing circuits.



Eight headline results. Orange: before this work. Blue: now. Every number is from a simulation and is explained on the page for that step.

The design flow, end to end



Blue boxes are new or substantially improved. You give a spec; SpiceMCP picks a known circuit, sizes it, tunes it in the simulator, turns it into orderable parts, checks how many built boards would pass, and draws the schematic.

How to read this report

- One section per improvement. Each starts with the problem in one sentence, then a **before / after** picture, then what it means for your design.
- **Orange** is always the old behaviour, **blue** the new one, a sand-coloured band is the range the spec allows.
- No simulator knowledge is needed. The few terms that matter (standard values, tolerance, yield, margin) are explained on the last page.

STEP 1 OF 10

Easier to drive: fewer, clearer controls

SpiceMCP is operated through an AI assistant, which picks from a list of commands. The old list was long and loosely described, so the assistant guessed. Guessing is where wrong circuits come from.



Left to right: number of commands, size of the command list, longest description, inputs the assistant had to guess the shape of.

What changed

- 26 commands became 11 (13 today, after steps 3 and 5 added two). Each has a short, strict description and typed inputs.
- Every answer ends with **the next step to take**; every error says how to fix it and shows a correct example.
- The simulator's memory is the record: a past design is always restored from what was simulated, never retyped from memory.

What it means for you

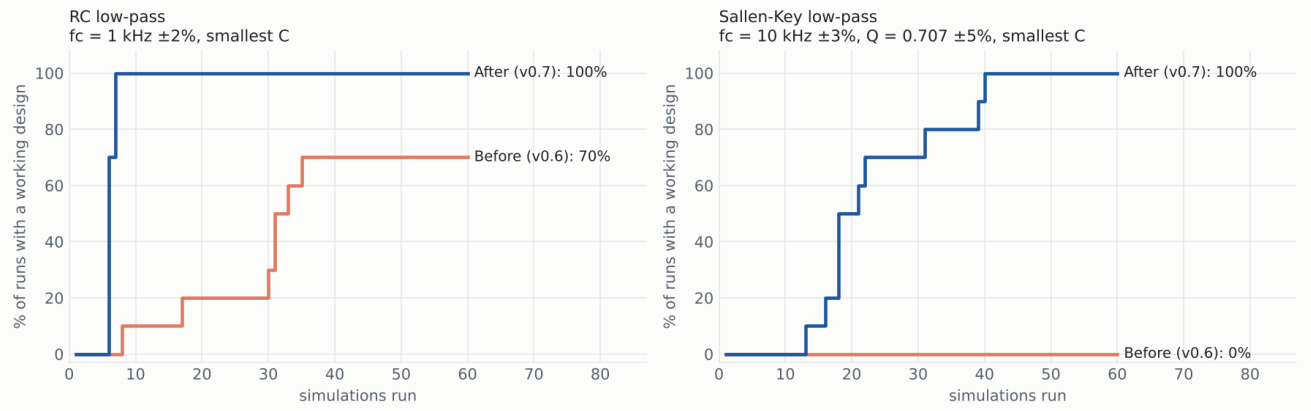
Fewer wrong turns and fewer silently wrong circuits, also with smaller, cheaper AI models.

STEP 2 OF 10

Finds working part values reliably

Part values span decades (100 Ω to 1 MΩ, 1 pF to 1 µF), and several specs pull on the same parts. The old search stepped in straight lines and adjusted one spec at a time, so it often got stuck.

How often the optimizer finds a design that meets every spec (10 runs each)



Share of 10 independent runs that had found a design meeting every spec, against the number of simulations spent.

problem	runs that met every spec	simulations to first working design
RC low-pass, fc = 1 kHz ±2%	7 of 10 → **10 of 10**	31 → **6**
Sallen-Key, fc = 10 kHz ±3% and Q = 0.707 ±5%	0 of 10 → **10 of 10**	never → **20**

What changed

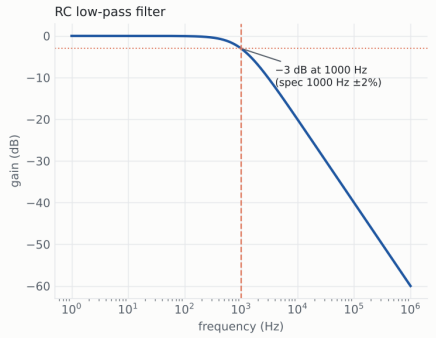
- Steps are taken in ratios (×1.2, ÷1.5), the way an engineer thinks about part values.
- It first samples the whole range evenly, so it does not start in a corner.
- When several specs pull on the same parts (a filter's corner and its Q), it solves for all of them together, from measured sensitivities, instead of one at a time.

STEP 3 OF 10

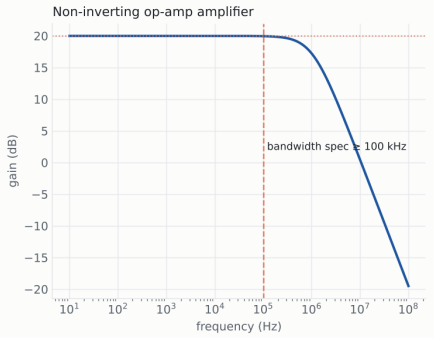
Start from a known circuit, sized by textbook equations

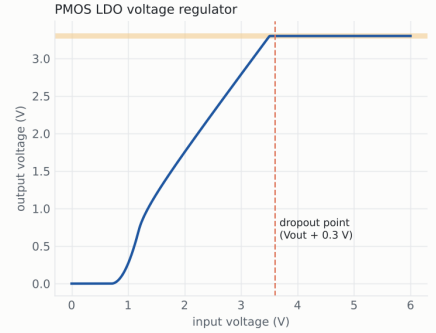
Engineers rarely start from a blank page. Now you can name a circuit and a spec, and get a netlist sized by the standard design equations, already checked in simulation.

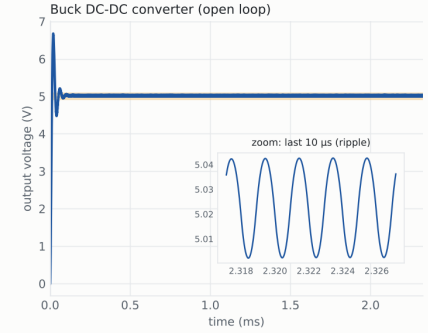
Topology library: five circuits, sized by textbook equations, then simulated









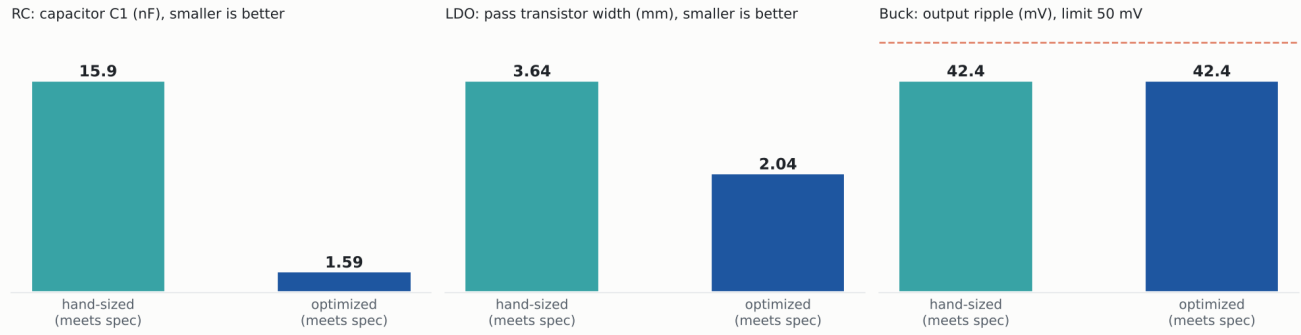


Spec vs simulated, hand-sized values (no optimization)

circuit	metric	spec	simulated
RC low-pass	fc	1000 Hz	1000.0 Hz
Sallen-Key	fc (−3 dB)	10000 Hz	9998 Hz
Sallen-Key	gain at fc/2	≥ −0.5 dB	-0.24 dB
Amplifier	gain	20.0 dB (10×)	20.00 dB
Amplifier	bandwidth	≥ 100 kHz	1060 kHz
LDO	Vout at 5 V in	3.30 V ±1%	3.301 V
LDO	Vout at 3.6 V in	≥ 3.234 V	3.300 V
Buck	Vout	5.0 V ±2%	5.023 V
Buck	ripple	≤ 50 mV	42.4 mV
Buck	efficiency	(reported)	94.6 %

All five ready-made circuits at their hand-calculated values, simulated. The table (bottom right) compares spec and simulation.

The optimizer refines the hand-sized start (40 simulations each)



Tuning then improves the hand-sized start: a 10× smaller capacitor in the RC filter, 14% smaller C1 in the Sallen-Key, a 44% smaller pass transistor in the LDO.

What the five circuits are

circuit	you give	sized by
RC low-pass	corner frequency	fc = 1 / (2πRC)
Sallen-Key low-pass (2nd order)	corner frequency, Q	equal-R design: C1 = 2Q/(ωR), C2 = 1/(2QωR)
Non-inverting amplifier	gain, bandwidth, op-amp GBW	R2 = R1·(G−1); checks GBW ≥ G·bandwidth
PMOS LDO regulator	Vin, Vout, load current, dropout	divider from Vref; pass device from current and dropout
Buck converter	Vin, Vout, Iout, switching frequency, ripple	duty = Vout/Vin; L from ripple current; C from ripple voltage

What it means for you

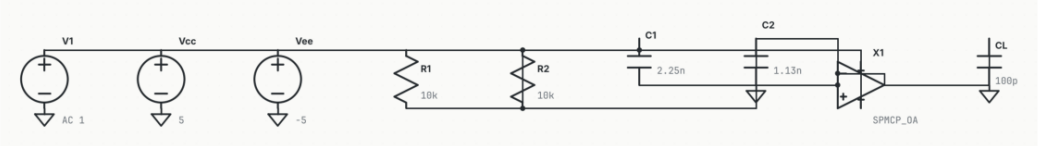
- A working start in one step, with the hand calculation shown line by line, so you can check the reasoning.
- Impossible specs are refused up front with the reason (for example, a gain-bandwidth product the op-amp cannot reach, or a dropout below what the pass device allows).
- Tuning starts close to the answer, so it spends its simulations improving the design rather than searching for one.

STEP 4 OF 10

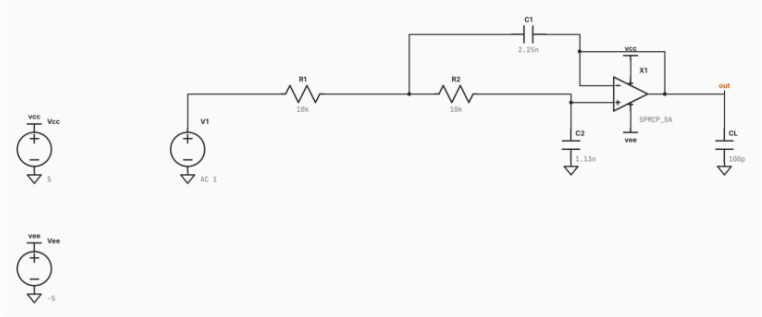
Schematics drawn automatically, and proven correct

A netlist is a list of connections, not a picture. Before, without someone placing every part by hand, parts landed in a row and straight wires joined them. In the simulator, a wire that crosses a pin connects to it, so those drawings were quietly a **different circuit**.

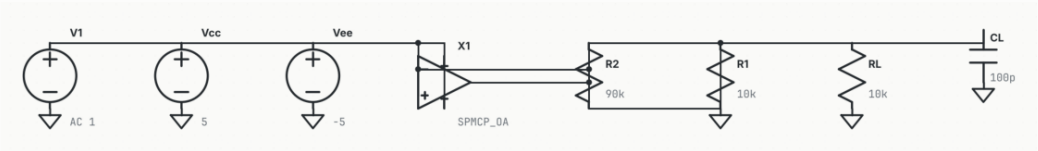
Sallen-Key low-pass filter. BEFORE: parts in a row. wrong circuit: 20 node(s) shorted or broken



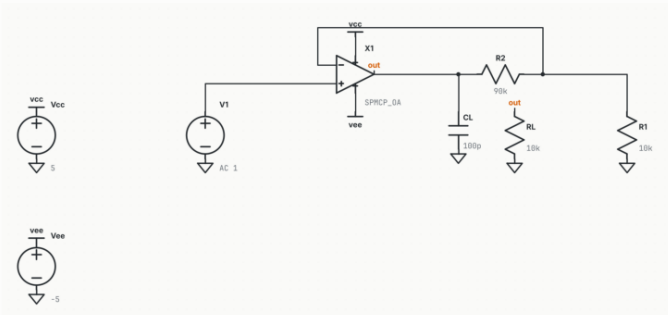
Sallen-Key low-pass filter. AFTER: drawn automatically. same circuit as the netlist



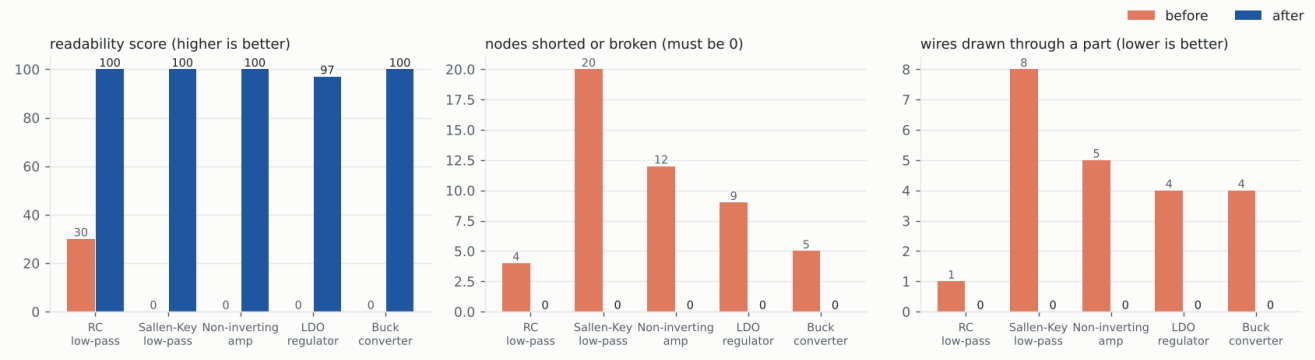
Non-inverting amplifier. BEFORE: parts in a row. wrong circuit: 12 node(s) shorted or broken



Non-inverting amplifier. AFTER: drawn automatically. same circuit as the netlist



Orange title: the old way. Blue title: drawn automatically. Each title says whether the drawing is still the circuit in the netlist. In the old drawings, look for wires running straight across resistor and capacitor pins.



circuit	before: score	before: shorted or broken	step 4: score	step 4: shorted or broken	today (v0.15.0): score
RC low-pass	30	4	**100**	**0**	**100**
Sallen-Key low-pass	0	20	**100**	**0**	**100**
Non-inverting amp	0	12	**100**	**0**	**100**
LDO regulator	0	9	**97**	**0**	**76**
Buck converter	0	5	**100**	**0**	**69**

Readability score: 100 minus points for each wire crossing, wire through a part, overlapping part, hidden connection and dangling wire. The step 4 columns were measured when step 4 shipped. The last column is measured from today's drawings every time this PDF is built: the LDO and buck gained a real op-amp (step 6) and a controller (step 8), which added parts and lowered their scores.

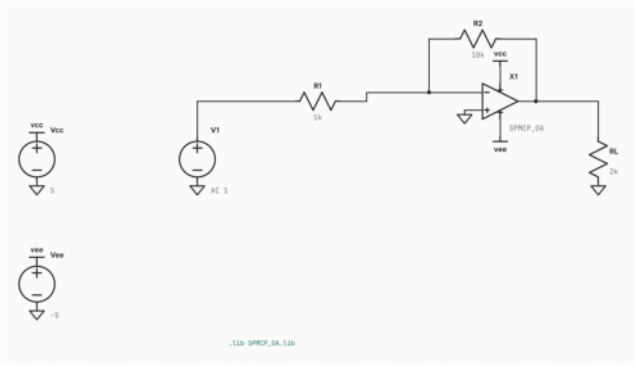
How the drawing is laid out

- Signal flows left to right; an amplifier's inputs are always left of its output.
- Ground at the bottom: every grounded part hangs down and gets its own ground symbol.
- Feedback parts (a feedback resistor, a Sallen-Key's C1) get their own lane above or below the main path. Transistor amplifiers stand upright, drain above source.
- Wires are right-angled, never pass through a part, and never touch another net. Measured nodes (out, fb, ref) are labelled.
- **Every drawing is read back and compared with the netlist, pin by pin.** A drawing that is not the same circuit is refused, never shown.

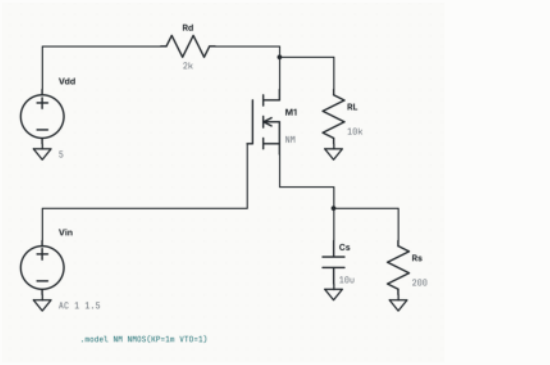
It works on circuits it was never tuned for

The layout rules were written against the five circuits above. These were tried only afterwards (scores 99 to 100, no crossings):

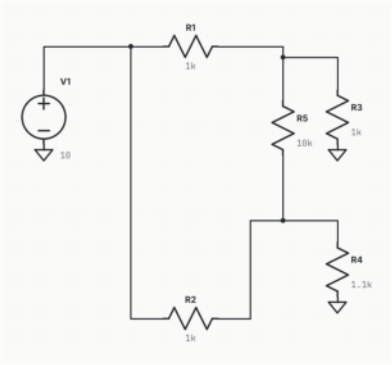
Inverting amplifier · score 100/100 · 0 crossings



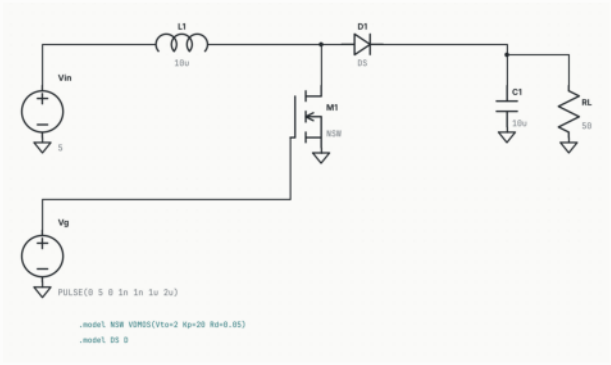
Common-source MOSFET amp · score 99/100 · 0 crossings



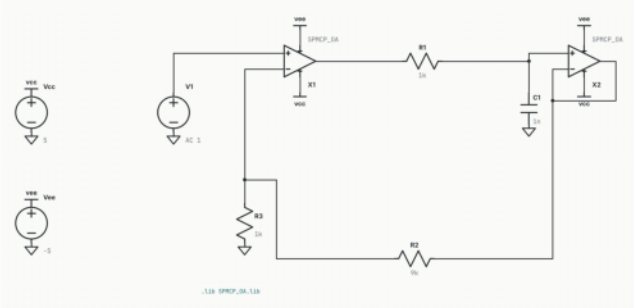
Wheatstone bridge · score 99/100 · 0 crossings



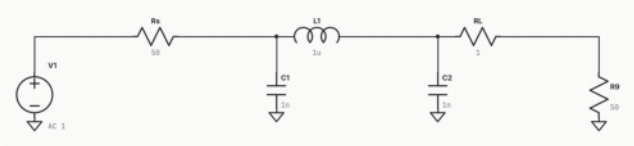
Boost converter · score 100/100 · 0 crossings



Two-stage amp with feedback · score 100/100 · 0 crossings



Pi filter · score 100/100 · 0 crossings



STEP 5 OF 10

From ideal values to a parts list you can order

Tuning produces values like C1 = 1.5915 nF. Nobody sells that. Rounding every part to the nearest standard value moves all of them at once, and on a real board every part is also off by its tolerance. A design that sits on the edge of its spec passes on paper and fails on many boards.

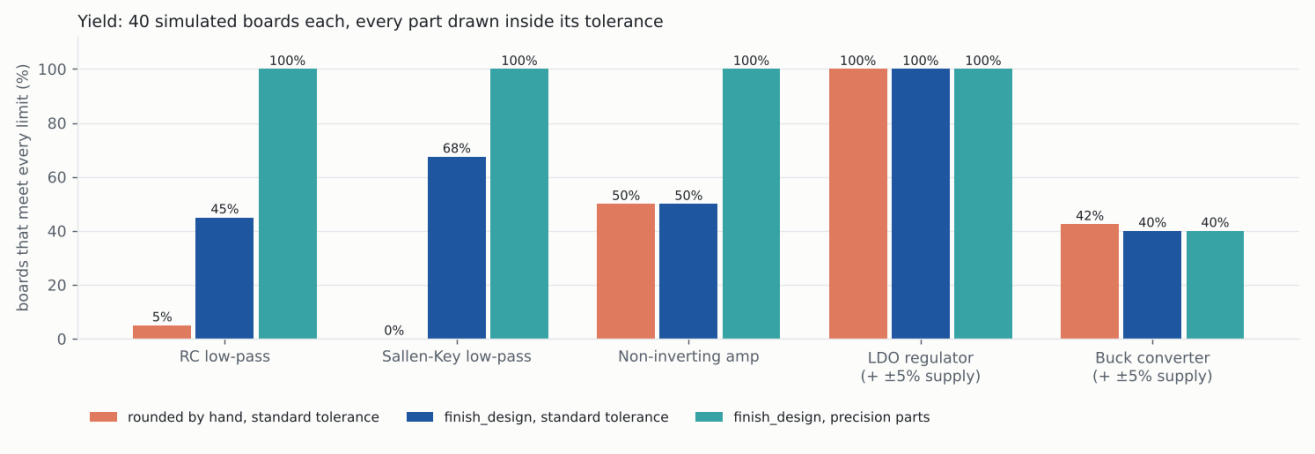
circuit	part	optimiser value (not a real part)	rounded by hand (nearest standard)	SpiceMCP finish_design
RC low-pass	C1	1.59n	1.5n	1.8n (E12, ±5%)
	R1	100k	100k	88.7k (E96, ±1%)
		limits met?	NO	YES
Sallen-Key low-pass	C1	1.95n	1.8n	2.2n (E12, ±5%)
	C2	1.13n	1.2n	1n (E12, ±5%)
	R1	10k	10k	13.3k (E96, ±1%)
	R2	10k	10k	9.31k (E96, ±1%)
		limits met?	NO	YES
Non-inverting amp	R1	10k	10k	10k (E96, ±1%)
	R2	90k	90.9k	90.9k (E96, ±1%)
		limits met?	YES	YES
LDO regulator	R1	175k	174k	174k (E96, ±1%)
		limits met?	YES	YES
Buck converter	C	1.88u	1.8u	2.2u (E12, ±5%)
	L	20.3u	22u	22u (E12, ±20%)
		limits met?	YES	YES

Rounding by hand breaks the two filters: the corner frequency leaves its window. The new finish step finds standard-value sets that meet every limit for all five.

How the finish step works

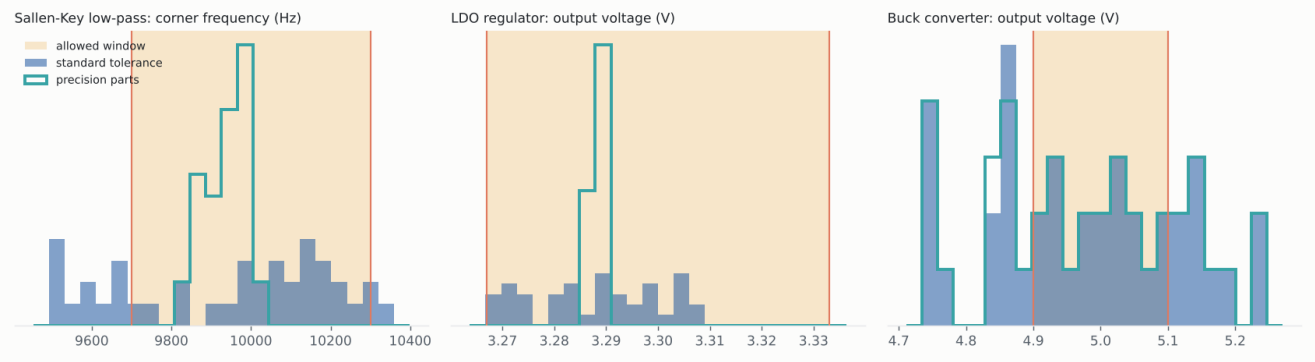
- Pick the coarse-step parts first (capacitors come in E12: 1.0, 1.2, 1.5, 1.8, 2.2 ...), trying the standard value just below and just above.
- Re-centre the fine-step parts (1% resistors in E96) so the design sits in the middle of its limits, the way you would recompute R after choosing C.
- Build 30–40 virtual boards with every part at a random value inside its tolerance, plus the supply within ±5% where it matters, and keep the set that passes most often.

Yield: how many built boards would meet the spec



circuit	rounded by hand	finish step	finish step, precision parts
RC low-pass	5%	**45%**	100%
Sallen-Key low-pass	0%	**68%**	100%
Non-inverting amp	50%	**50%**	100%
LDO regulator (+ ±5% supply)	100%	**100%**	100%
Buck converter (+ ±5% supply)	42%	**40%**	40%

Standard tolerance: 1% resistors, 5% capacitors, 20% inductors. Precision parts: 0.1% resistors, 1% capacitors, 10% inductors. Both columns use the same virtual boards, so the comparison is fair; 40 boards resolve yield to about ±8%.



Where the boards land. Shaded band: allowed window. Filled: standard tolerance. Outline: precision parts.

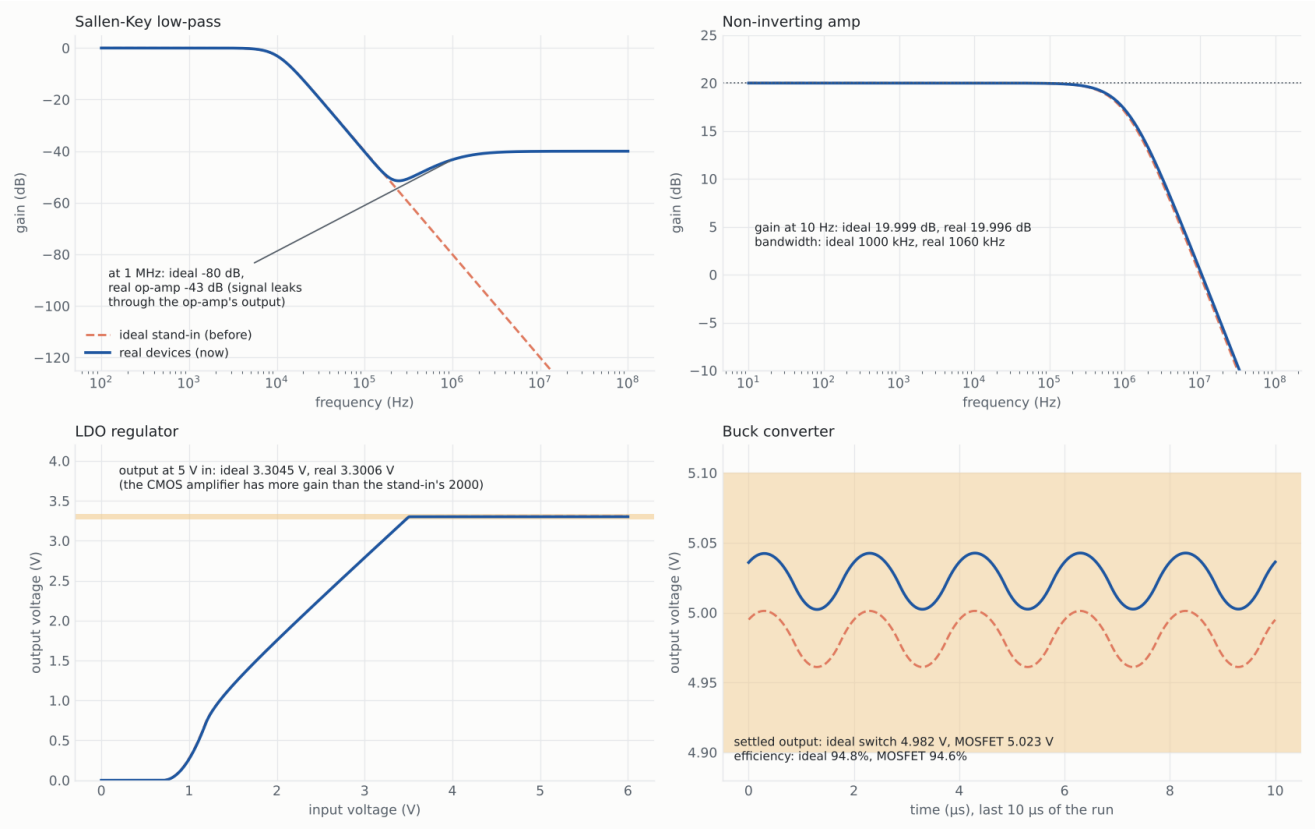
What the yield numbers tell a designer

- **Filters.** A ±2–3% corner frequency cannot be held with 5% capacitors, whatever values you choose. The finish step gets the most out of them (Sallen-Key 0% → 68% of boards), and 1% capacitors take it to 100%.
- **Amplifier.** ±0.1 dB of gain is ±1.2%. Two 1% resistors can be 2% off in ratio. Use 0.1% resistors (100%) or loosen the spec.
- **LDO.** Its feedback loop holds 3.3 V against ±5% supply and 1% resistors: 100%.
- **Buck.** This ready-made buck runs without feedback, so its output follows its input: ±5% in gives ±5% out against a ±2% spec. Better parts do not help (40%). The fix is a control loop, and the yield check shows that before a board is built.

STEP 6 OF 10

Real op-amps and switches, not ideal stand-ins

The amplifier circuits used to model the op-amp as a dependent voltage source: one number for its gain, no supply, no limits. The buck used an ideal switch. Quick for a first look, but it hides what a real part does.



Same circuits, same values. Dashed orange: the ideal stand-in used before. Solid blue: real devices, used now.

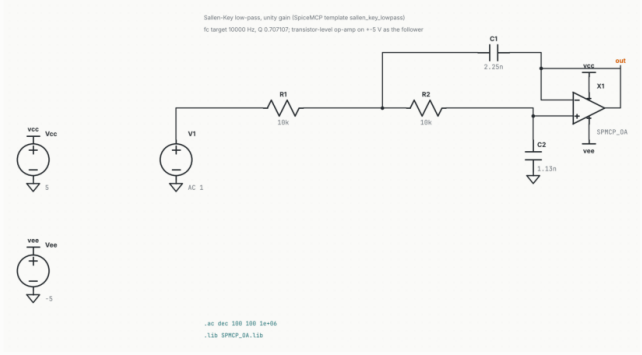
What replaced the stand-ins

- **Op-amp** (Sallen-Key, amplifier, LDO error amplifier): a two-stage CMOS op-amp made of eight MOSFETs, running from its own supply pins. Measured: 87 dB open-loop gain, 10.6 MHz gain-bandwidth, 10 V/µs slew, 1.1 mA supply current. It has the standard 5-pin layout, so a vendor model can be swapped in.
- **Switch** (buck): a P-channel power MOSFET (50 mΩ, 1 nF gate) with a gate driver.

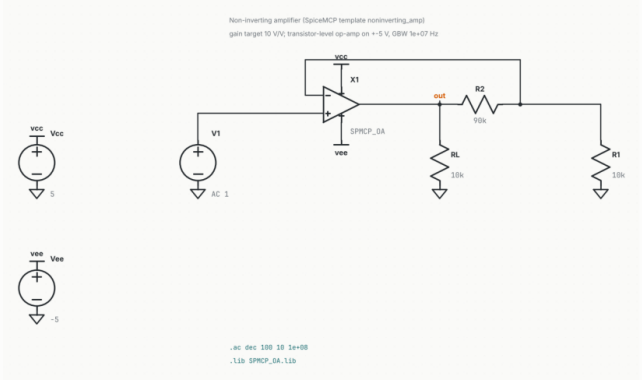
What the real parts show that the stand-ins hid

- **Sallen-Key.** At 1 MHz the ideal model claims -80 dB. The real op-amp gives -43 dB, because signal leaks through the feedback capacitor into the op-amp's output. If the filter must block a clock or switching noise, only the real model gives the right answer.
- **Amplifier.** Gain and bandwidth agree within a few percent. The real op-amp adds what the stand-in left out: supply current, output swing and slew rate.
- **LDO.** Both regulate. The real amplifier has more gain, so the output is closer to 3.3 V (3.3006 V vs 3.3045 V).
- **Buck.** The MOSFET's gate charge stretches every on-time slightly: 4.98 V becomes 5.02 V. The sizing equations now keep 20% ripple margin for it.
- All five ready-made circuits still meet their spec at the hand-sized start.

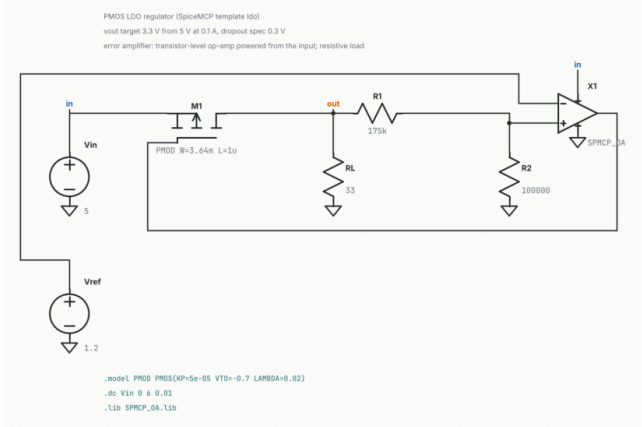
Sallen-Key low-pass · readability 100/100



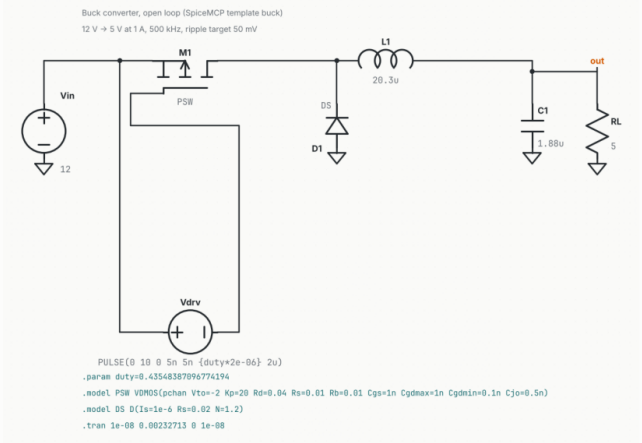
Non-inverting amp · readability 100/100



LDO regulator · readability 97/100



Buck converter · readability 100/100



The schematics now show the real parts: the standard op-amp symbol with supply labels, and the power MOSFET. The op-amp's circuit is saved next to the drawing, so it opens and simulates in LTspice as it is.

STEP 7 OF 10

Proven in LTspice on Windows, and checked on every change

Until now every result came from ngspice on Linux. The tool is meant for LTspice on Windows, so it had to be shown working there, on a real install.

circuit	measurement	LTspice 26 (Windows)	ngspice 42	difference	meets spec in LTspice
RC low-pass	corner frequency	1.0000 kHz	1.0000 kHz	0.00%	YES
Sallen-Key low-pass	corner frequency	9.9977 kHz	9.9978 kHz	0.00%	YES
	gain at 10× fc	-40.2568 dB	-40.2568 dB	0.00%	
Non-inverting amp	gain	19.9962 dB	19.9962 dB	0.00%	YES
	bandwidth	1.0599 MHz	1.0600 MHz	0.00%	
LDO regulator	output at 5 V in	3.3006 V	3.3006 V	0.00%	YES
	output at dropout	3.3003 V	3.3003 V	0.00%	
Buck converter	output voltage	5.0204 V	5.0231 V	0.05%	YES
	ripple	41.5936 mV	42.4330 mV	1.98%	
	efficiency	94.5%	94.6%	0.06%	

The five ready-made circuits, simulated in LTspice (Windows 11, LTspice 26) and in ngspice from the same files. They agree within 2%; the largest gap is the buck's ripple, under 1 mV on a 42 mV figure.

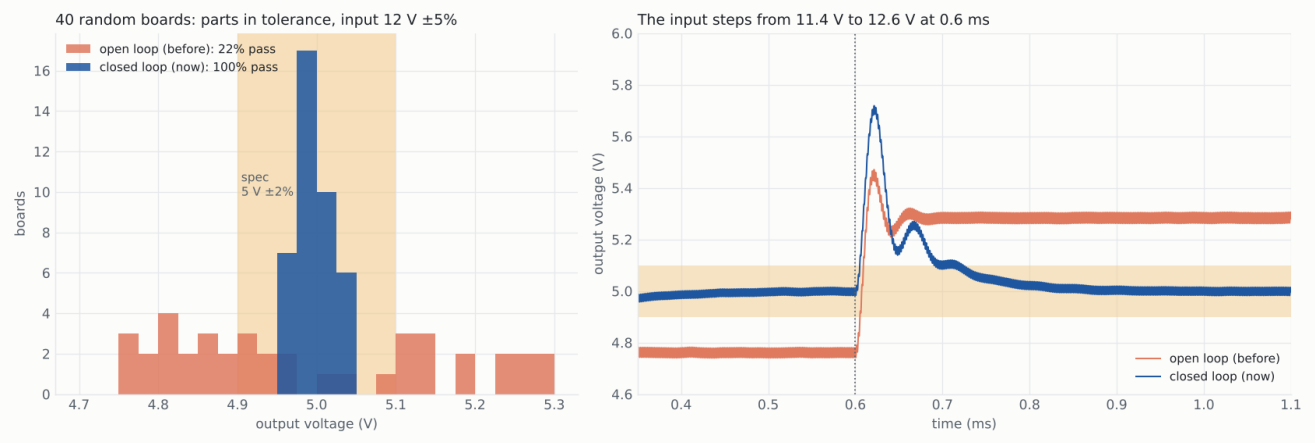
What was checked in LTspice

- **Schematics.** 13 of 13 drawn schematics were read back by LTspice itself and connect exactly the same pins as the circuit they were drawn from. That covers the symbols, the op-amp library file and every wire.
- **Circuits.** All 5 ready-made circuits run in LTspice and meet their spec at the hand-sized start.
- **Reference files.** 10 of 10 LTspice result files were produced and are now stored with the project, so the waveform readers are tested against real LTspice output.
- **Automatic checks.** Every change now runs the full test suite on Windows and on Linux, with two Python versions each, before it can be merged.
- **Version numbers.** Semantic versioning: 0.12.0 now. 1.0 will mean the tool controls are frozen, so scripts written against it keep working.

STEP 8 OF 10

A buck that regulates, and every feedback loop checked for stability

The buck ran at a fixed duty cycle, so its output followed the input. And a circuit with feedback can meet every spec on paper and still oscillate on the bench.



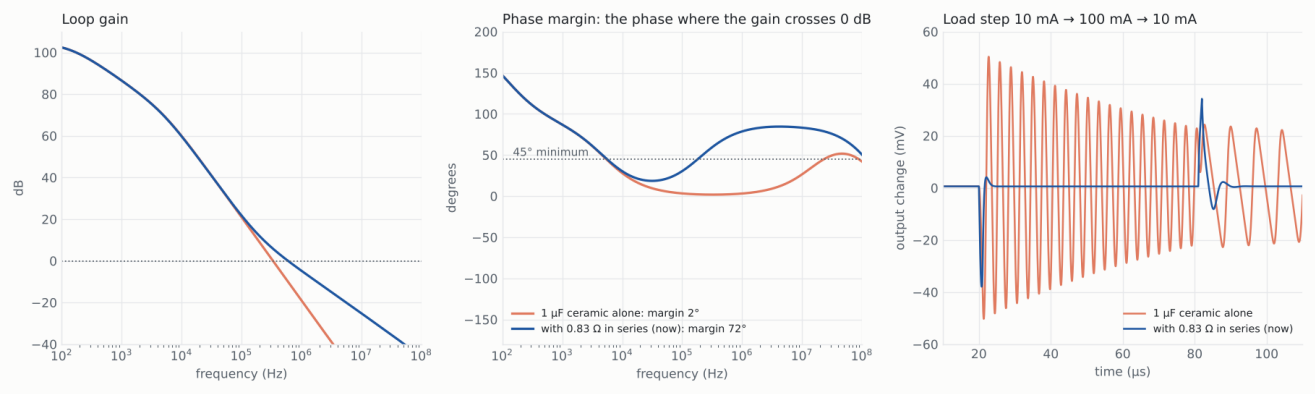
Left: the same 40 random boards (parts in tolerance, input 12 V ±5%). Right: the input jumps by 1.2 V at 0.6 ms.

The buck now has a controller

- An error amplifier, a comparator against a ramp and a gate driver, all built from transistors, set the duty cycle. The output no longer follows the input.
- **Yield 22% → 100%.** Open loop, the boards spread from 4.77 to 5.28 V. Closed loop, from 4.96 to 5.03 V, set by the 1% divider resistors.
- When the input jumps, the output peaks at 5.7 V and is back in spec after 124 µs. The loop is deliberately slow and simple; a faster compensator would cut that peak.
- Efficiency now counts the real gate drive and controller current: about 89%.

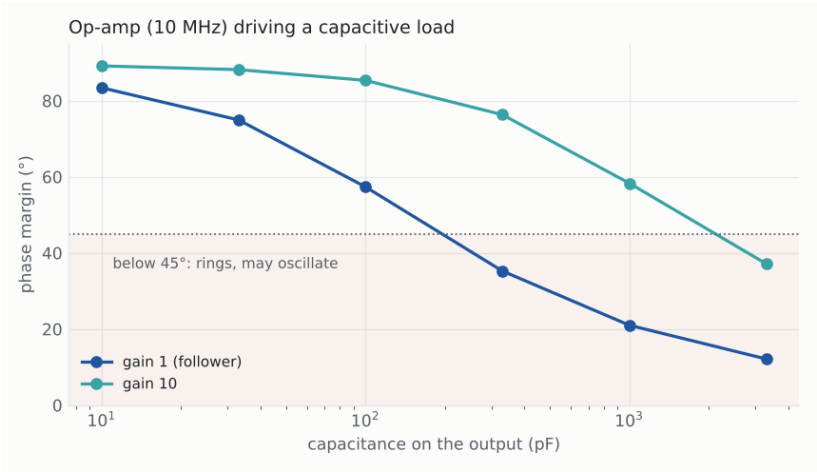
Stability, checked every time a circuit is simulated

Each feedback circuit carries a second simulation with its loop cut open at the amplifier input. It measures the **phase margin**: how far the loop is from oscillating. 45° is the usual minimum; below it a circuit rings, near 0° it oscillates. The optimiser and the yield run see it like any other spec.



LDO with the 1 µF output capacitor every regulator needs. Orange: the capacitor alone. Blue: with the small series resistor the design now sizes.

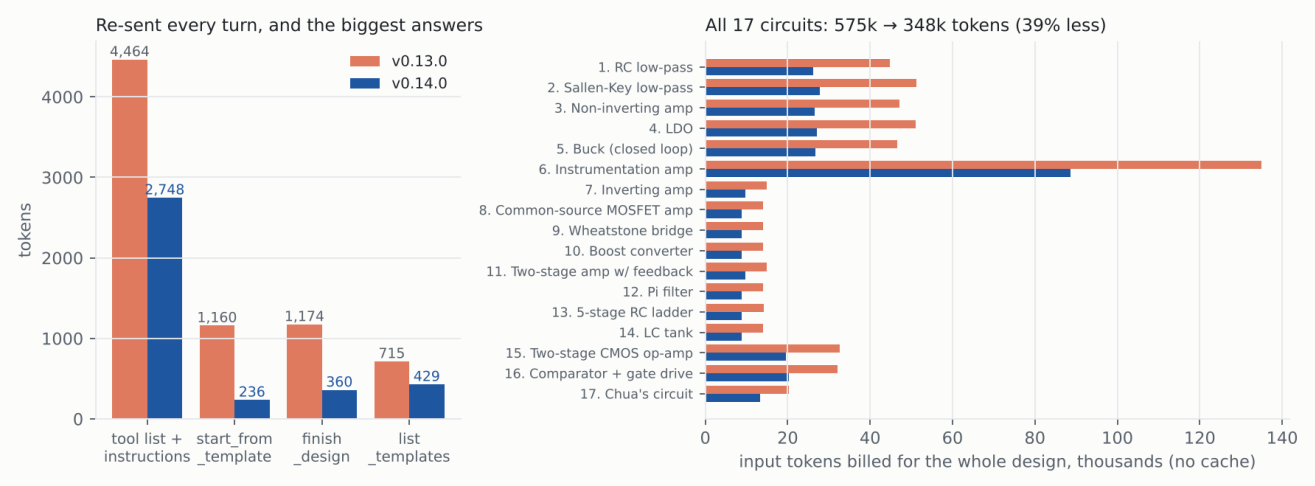
- **LDO.** With a ceramic capacitor alone the phase margin is 2°: a load step rings for the whole pulse, and a board would oscillate. A 0.83 Ω resistor in series with the capacitor (or a capacitor with that ESR) gives 72°, and a 39 mV dip on a 10× load step that settles in about 1.5 µs. The resistor is now in the parts list.
- **Op-amp circuits.** The op-amp's transistors now have their capacitances, so its phase margin falls as the capacitance it drives rises, as on a real part. A follower drops below 45° at about 200 pF; at a gain of 10 it holds to about 2 nF. The filter and the amplifier, at 100 pF, have 86° and 85°.



STEP 9 OF 10

Cheaper to run, nothing misleading, and licensed

Everything SpiceMCP says is read by an AI assistant, and every word costs money on every turn. The command list alone was re-sent on each turn. And one sentence in the yield answer claimed more certainty than the simulation gave.

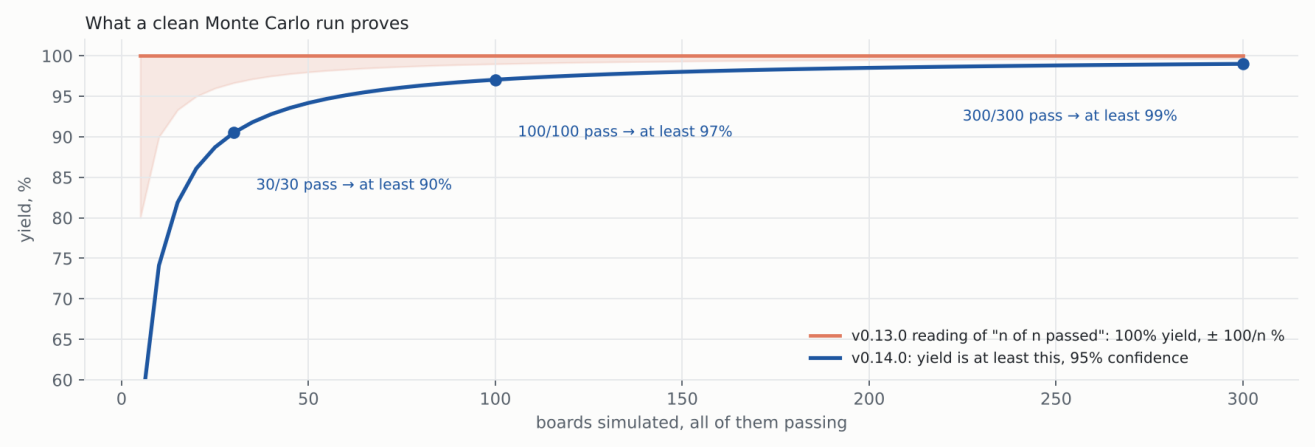


Left: the part re-sent on every turn, and the three biggest answers, in tokens (the unit an assistant is billed in). Right: the whole design of each of the 17 circuits in the project, the same flows before and after.

What changed

- **Command list 4,464 → 2,748 tokens**, re-sent on every turn: automatic labels removed, field text shortened, expert settings still accepted but no longer listed.
- **Answers are one line, numbers to 6 digits.** Indentation alone was a fifth of every answer.
- **Short answers, details on request.** Starting a ready-made circuit answered in 1,160 tokens on average, now 236; the parts list in 1,174, now 360. The hand calculation, warnings and alternative part sets are one call away.
- **All 17 circuits: 575k → 348k input tokens** without caching (39% less), 168k → 103k with caching.
- **MIT license.** Without one, nobody could legally use the code at work.

An honest yield



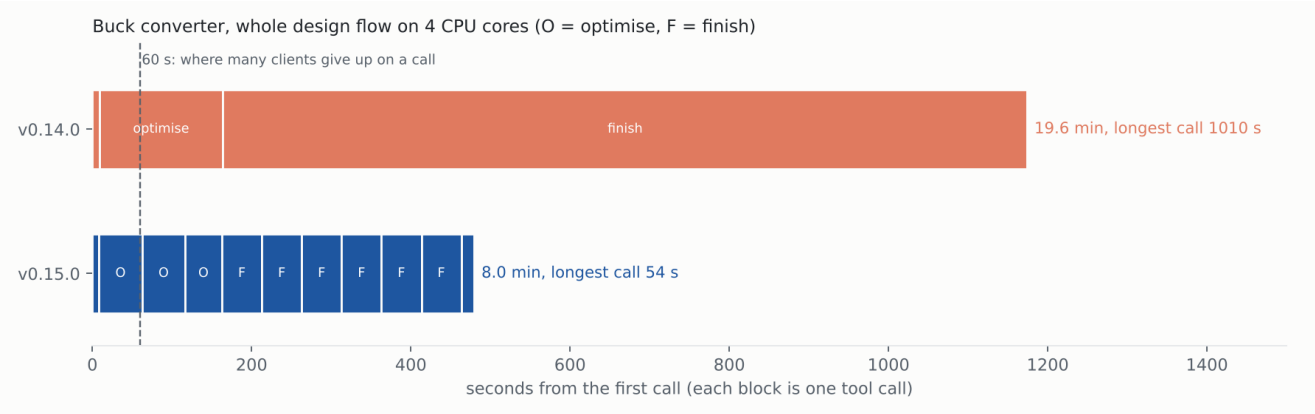
Orange: how a clean run used to read. Blue: what it actually proves.

- Before: "30 runs resolve yield to about 3%". That is wrong. 30 boards out of 30 passing does not show the yield is 97 to 100%; a design with 92% yield passes all 30 about one time in ten.
- Now: "30/30 pass: yield is at least 90% (95% confidence)". This is the standard lower bound (Clopper-Pearson). 100 clean boards prove 97%, 300 prove 99%.
- Each yield run now draws new boards. It used the same random boards every time, so a re-run could never find a failure the first run missed. The seed is reported, so any run can be repeated exactly.
- This report itself is rebuilt: the copy shipped with 0.13 was two files written over each other and would not open. It is now written in one piece, and a test opens every shipped PDF.

STEP 10 OF 10

No more waiting minutes for one answer

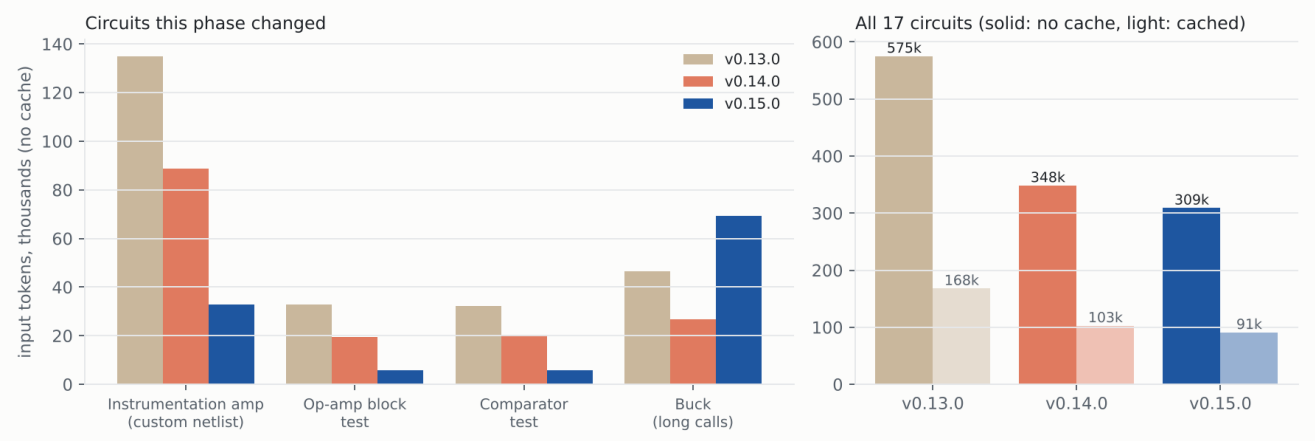
Designing the buck converter took 20 minutes, 17 of them inside one call. Most assistant apps give up on a call after about a minute, so the design was lost even though the work was done.



Every tool call of one buck design, end to end, on a 4-core machine with ngspice. Each block is one call.

What changed

- **No call runs longer than about a minute** (longest 54 s, was 1010 s). Tuning stops at the limit with its progress saved and continues on the next call. Turning the design into parts keeps working in the background; asking again picks up the result. Both send progress updates.
- **20 → 8 minutes for the buck.** Simulations that do not depend on each other now run side by side, one per processor core: the yield boards, the standard-value combinations, a circuit's stability and load-step checks. On an 8-core laptop it is faster still.
- **Built-in op-amp and comparator by name.** A circuit can use them without copying their transistor model in: the instrumentation amplifier design dropped from 89k to 33k tokens.
- **Works without LTspice.** One-off simulations and sweeps follow the chosen simulator, so macOS and Linux users with ngspice can use every command.
- **Ready for pip install.** Packaged for the Python package index, checked in every build, and tested with both the current and the new version of the MCP library.



All 17 circuits: 575k → 309k input tokens without caching. The one that went up is the buck: shorter calls mean more of them, and each re-reads the conversation. With caching that costs little (light bars); clients that tolerate long calls can raise the time limit.

Where it stands, honestly

- Results were simulated in ngspice and confirmed in LTspice on Windows: the two agree within 2%, so they do not depend on one tool.
- The automatic schematic handles flat circuits and 5-pin op-amps. Other sub-blocks (subcircuits) are refused with a clear message rather than drawn wrongly.
- The transistor-level op-amp uses simple MOSFET models. Its phase margin is now measured, but for a stability sign-off use the vendor model of the part you will buy.
- Yield covers part tolerance and supply. Temperature and silicon process corners need real device models; the design report can run them where models are supplied.
- Test suite: about 340 automated checks, run on Windows and Linux for every change.

Five terms, in plain words

term	meaning
Standard values (E-series)	The values parts are actually sold in. E12 has 12 per decade (1.0, 1.2, 1.5 ... 8.2), E96 has 96 (1.00, 1.02, 1.05 ...).
Tolerance	How far a real part may be from its marked value: a 5% 10 nF capacitor is somewhere between 9.5 and 10.5 nF.
Yield	The share of built boards that meet every spec, with every part somewhere inside its tolerance.
Margin	How far the design sits inside its nearest limit. Zero means on the edge: half the boards would fail.
Netlist	The text list of which part connects to which node: what the simulator reads, and what the schematic must match.